

What should I wear — delivered in 45 minutes.

Pilot Launch Strategy

The leanest path to a live, paid order in Nagpur — what ships, what becomes manual ops, and what waits.

Prepared for the founding team · 9 August 2026

Scope: product, engineering, operations, and compliance for a 4-neighbourhood Nagpur pilot

“Launch is a supply-and-operations test, not a product test. The job is to prove fifty Nagpur shopkeepers will keep stock accurate and that a 45-minute promise survives contact with real traffic — not to ship every feature this app could eventually have.”

~10–12	~35+	4	45
weeks to pilot (vs. 22 full-scope)	launch-blocking items shipped	neighbourhoods at launch, not the city	minute delivery promise, enforced

Contents

- 1 The governing principle
- 2 What changed from the full 22-week plan, and why
- 3 Launch scope — customer app
- 4 Launch scope — vendor app
- 5 Launch scope — backend & platform

~~6 Launch scope — admin panel~~

- 7 Launch scope — design identity
- 8 Launch scope — release & compliance
- 9 Deferred to manual operations, with rationale
- 10 Deferred outright
- 11 Capture the data now, even though the feature waits
- 12 The multi-vendor cart decision
- 13 External dependencies — what to skip, what to start today
- 14 The escalation ladder, resolved
- 15 Week-by-week phase plan
- 16 Risk register
- 17 Post-pilot: what gets built next, and in what order

1. The governing principle

Every feature in the original 110-item backlog is real work the app will eventually need. Not all of it needs to exist before the first paid order. This document applies one rule consistently, and every scope decision below traces back to it:

Defer the feature. Capture the data.

Two tests decide whether an item ships at launch or moves to Section 9 or 10:

- **Can a human do this correctly for the first ~500 orders?** At 50 shops and 20–40 orders a day, one attentive ops person watching a board and making phone calls is cheaper, more flexible, and — critically — more informative than an automated system built on guesses. The failure modes discovered by a human doing the job by hand become the actual specification for the automated version.
- **Does the feature's value depend on data the app doesn't have yet?** The “Kya Pehnu?” answer engine, fit certainty, and every retention mechanic need either accumulated order history or a catalogue thick enough that the results don't look broken. Fifty shops on day one cannot support them well regardless of how much engineering time is spent — the constraint is inventory density, not code.

Nothing on the deferred lists is cancelled. Sections 9 and 10 name, for every cut item, exactly what replaces it operationally and when the automated version gets built (Section 17). The one thing that is never deferred is the data those future features will need — see Section 11.

2. What changed from the full 22-week plan, and why

The original roadmap (see the team's earlier planning documents, Docs 00–06) sequenced nine phases across roughly 22 weeks and built every subsystem — the WhatsApp bot, the full Porter integration with live tracking, the escalation ladder through SMS and IVR, vendor settlements, and all eight differentiator features — before the first pilot order. That plan is correct as an eventual target. It is the wrong first step, because it spends ten weeks perfecting systems whose real requirements are still unknown.

This document proposes the same destination reached in roughly half the time, by replacing automation with a person wherever a person is cheaper and smarter at low volume, and by cutting one regulatory dependency (SMS/DLT) outright rather than working around it.

Dimension	Full-scope plan	Optimal pilot plan
Time to first paid pilot order	~13–15 weeks (Phases 0–1)	~5–6 weeks
Time to public pilot launch	~22 weeks	~10–12 weeks
WhatsApp vendor bot	Built before launch	Deferred — admin does listing #1–20 by hand
Porter integration	Full quote/track/webhooks before launch	Manual dispatch; interface built, adapter deferred
Escalation ladder	Push → WhatsApp → SMS → IVR → reassign	Push → WhatsApp → IVR → human (SMS/DLT cut)
Differentiator features	Built in Phase 6, before launch	All eight deferred past the pilot

Dimension	Full-scope plan	Optimal pilot plan
Admin panel scope	13 full modules	4 modules: QC, order board, catalogue, refunds
External accounts needed pre-launch	7 (Meta, Porter, DLT, Razorpay, Apple, Google, GST)	3 (Google Play org, Apple org, GST/entity)

The full-scope plan is not abandoned — it becomes Section 17, the post-pilot build order, now sequenced by evidence from the pilot instead of by upfront guesswork.

3. Launch scope — customer app

The customer app has never called the backend — it runs entirely on `mockStores.js`. This section is not additive polish; it is the app becoming real for the first time. Nothing below is optional if a stranger is going to pay real money for a real shirt.

3.1 Identity and account

- Phone number + OTP login (primary, matches how India expects to sign in), Google sign-in as a secondary path.
- Session persistence across app restarts, silent token refresh, clean sign-out.
- In-app account deletion, plus the public web page both stores require alongside it.

3.2 Browsing and product data

- Real API wiring for nearby shops, product list, and product detail — the mock data layer is deleted, not just supplemented.
- Product detail: real images, sizes and live stock, price, and shop context.

Search, filters, sort, and category browsing are explicitly out of launch scope (Section 10) — with 50 shops and a handful of categories each, a flat scrollable feed is not a degraded experience, it is the right experience.

3.3 Checkout

- Address flow: draggable map pin seeded from GPS, house/floor/landmark fields, a receiver name and phone that may differ from the account holder's.
- **Serviceability check before payment** — a customer outside the pilot polygon is told so before they reach for their card, never after.
- Payments: UPI intent (the dominant method in India) and Cash on Delivery with a value cap and a refused-delivery counter that disables COD per user past a threshold. Cards/netbanking/wallets follow once Razorpay production keys land (Section 13); Razorpay *test* keys are sufficient for everything up to that point.
- Order confirmation and an order detail screen the customer can return to.

3.4 The live order

- One screen, reachable in one tap from anywhere while an order is active: status, the vendor's shop, and — because Porter's live tracking is deferred to Section 9 — an honest status string driven by the ops board rather than a fabricated map animation. The current app simulates a driver icon moving along a fake route; that simulation is worse than no map at all and is removed, not shipped.
- Cancellation, with a clear cutoff (before the vendor marks ready) and an automatic refund.
- Push notifications at every real status change, with a deep link straight into this screen.

3.5 The states nobody designed yet

- Empty states ("Nothing here yet—Nagpur is stitching"), loading skeletons, error states with a retry action, and a first-run explanation of what the app is and how the 45-minute promise works.

These are not cosmetic. An app with no error state is an app that appears broken the first time the network hiccups — which, on a live pilot, is often.

3.6 Explicitly deferred from this app

Search/filters/sort, saved-address book UI, reorder, ratings and reviews, shop profile pages, wishlist, follow-a-shop, customer-facing Hindi/Marathi, a full dynamic-type accessibility pass, and every one of the eight differentiator features (answer engine, Try Rack, fit profile, wardrobe, occasion carts, Nagpur Now, emergency mode). Full list and reasoning in Sections 9–10.

4. Launch scope — vendor app

The vendor is the scarcer resource in this marketplace — there are always more willing buyers than willing, reliable shopkeepers in a new city. Every launch decision on the vendor side optimises for one thing: a vendor never misses an order because they didn't notice it.

4.1 What ships

- The existing accept / mark-ready flow, kept largely as-is — `transitionToReady`'s persist-then-dispatch design is already correct and is preserved.
- A loud, hard-to-miss alarm channel for new orders: a high-importance push notification channel with a distinct, looping sound and a full-screen swipe-to-accept card, so a phone in a pocket or on a counter still gets noticed.
- A one-tap “closed for now” toggle so a vendor stepping away doesn't have new orders routed to an empty shop.
- A product status field (`PENDING_QC` → `APPROVED` → `REJECTED`) so a listing created by admin on the vendor's behalf is visibly real to the vendor, even though they didn't create it themselves.

4.2 What is deliberately not built yet

- **The WhatsApp listing bot.** The plan's own operating guidance is to create a vendor's first ~20 listings for them by sending a person with a phone — which means the bot's first real user is listing #21, well past the pilot's opening weeks. Building it before there is a vendor who has outgrown manual onboarding is building for a user who doesn't exist yet.
- The escalation ladder's SMS and automated-voice rungs (Section 14) — replaced for now by push + WhatsApp + a human phone call.
- Sales dashboards, payout ledgers, and reliability scorecards — all become a weekly spreadsheet an ops person maintains by hand until there are enough vendors that a spreadsheet stops working.

***One thing not to cut:** vendor-facing Hindi/Marathi. The customer will tolerate an English-only app; the vendor is the scarce resource and the one who has to act correctly inside a 90-second window. Wherever the vendor app or its notifications use language, it ships bilingual from day one.*

5. Launch scope — backend & platform

This is where money, stock, and trust actually live, and it is the section with zero room to cut corners — every item here either prevents the platform from being trivially exploited or prevents an order from silently going wrong.

5.1 Correctness and money

- **Server-computed pricing object**, replacing the current single, client-trusted `totalPrice` field: items subtotal, platform commission, delivery fee charged, actual delivery cost, small-cart fee, surge, discount, GST, and a resulting contribution-margin figure. Computed from the catalogue at order time, on the server, and snapshotted onto the order so a later price or coupon change never rewrites history.
- **Transactional stock decrement** with a conditional update (so two concurrent buyers cannot both win the last unit) plus a 10-minute reservation at checkout and a release-on-timeout sweep for abandoned carts.
- **Idempotency keys** on checkout, so a double tap or a retried request cannot create two orders or two charges.

- Razorpay webhooks (signature-verified) as the source of truth for payment state — never the client's "payment succeeded" callback — plus a refund path wired to cancellations.

5.2 Logistics, kept honest

- A LogisticsProvider interface (quote, dispatch, track, cancel) with a **manual adapter** at launch: ops books a rider through Porter's own consumer app and pastes the tracking link into the admin order board. The interface exists so a Porter API adapter is a drop-in addition later, not a rewrite touching every place order state is read.
- This is a deliberate trade: it costs a small amount of ops attention per order in exchange for skipping Porter's 2–4 week commercial onboarding on the critical path, and for learning what the automated version actually needs to handle before building it.

5.3 Security — the part that is not negotiable

- **Fix the order-read authorisation hole immediately.** `getOrderById` currently checks nothing beyond a valid auth token — any signed-in user can read any order, including another customer's delivery address and phone number. This is a live bug in the current codebase, not a missing feature, and it is fixed in isolation, in the first commit of Phase 0, before any other work begins.
- Remove the `{ ...req.body }` spreads in the product controller — today a vendor request can set arbitrary fields on a product document, including ones a future field addition might not intend to be publicly writable.
- Zod (or equivalent) validation on every route boundary; nothing reaches Mongoose unvalidated.
- `helmet`, a real CORS allowlist (the mobile app is not a browser origin and needs none), and per-route rate limits — order creation, OTP requests, and webhooks each get their own limit tuned to their abuse shape.
- Webhook signature verification for every provider that sends one (Meta, Razorpay), raw-body capture where required, and dedupe by provider event id so a retried webhook cannot double-process.

5.4 Operability

- Boot-time environment-variable validation, so a missing secret fails the deploy instead of failing silently at the first customer order in production.
- Structured logging and Sentry error tracking on both the backend and the app.
- Nightly encrypted backups to object storage, **with the restore actually tested once** before launch — an untested backup is not a backup.
- A test suite covering pricing math, authentication/authorisation, and order status transitions specifically — the three places a silent bug costs real money or real trust — wired into CI so it runs on every change.

6. Launch scope — admin panel

The full plan specified thirteen admin modules. Four are load-bearing for a pilot; the rest are what a human with a spreadsheet does until volume makes the spreadsheet the bottleneck.

Module	What it does and why it ships
QC queue	Every listing — whether created by a vendor or by ops on a vendor's behalf — is approved or rejected here before it can appear in the customer app. This is the platform's only quality gate and it exists from day one.
Live order board	Every in-flight order as a card, coloured by how long it's been in its current status relative to the 45-minute promise. This is where the ops person actually works — watching the board, calling vendors, pasting Porter tracking links, and manually escalating anything that's stalling.
Catalogue editor with image upload	Lets ops create and edit a vendor's listings directly — this is the tool that does a vendor's first twenty listings, and it is what the WhatsApp bot eventually replaces.
Refund / cancel actions with an audit log	Every mutating action — approvals, refunds, cancellations — writes who did what and when. Non-negotiable the moment the panel can move money.

Vendor onboarding/KYC, the pricing-rules UI, the coupon builder, merchandising controls, feature flags, and the analytics dashboard are all deferred — see Section 9. Access control (Firebase custom claims, one role for the pilot) and the audit log are the only pieces of the eventual 13-module system pulled forward, because they cost little now and are much harder to retrofit onto actions that already happened.

7. Launch scope — design identity

The obsidian/charcoal/single-crimson palette already exists and is genuinely distinctive — it is kept exactly as is. The gap is entirely in typography and motion, and only the highest-leverage, cheapest pieces of that system ship for the pilot.

- A two-voice typography system: an editorial serif for chapter titles, product names and prices, and a neutral grotesk for everything functional. This is the single cheapest, highest-impact identity change available — the app currently ships no custom typeface at all.
- The token files (`typography.js`, `motion.js`, `haptics.js`) that make the system enforceable, following the same discipline already applied to `colors.js`.
- Two of the five signature animations: **the Unfurl** (product card expanding into the detail screen) and **the Thread** (the 45-minute countdown as a shortening hairline, the app's single most-seen and most memorable element). The remaining three — the Rail, the Fold, and the Window Light — wait for Section 17.
- Device-tier gating on the 3D scrollytelling scene, so it never threatens cold start on a low-end Android phone — the device the pilot's actual customers are most likely holding.
- A short copy pass across the screens that ship, in the voice already defined: short, confident, slightly editorial, never cute.

The Chapters tab, the full component gallery, and a complete accessibility audit are deferred — the identity work that matters for a first impression ships; the identity work that matters for a returning power user waits until there are returning power users.

8. Launch scope — release & compliance

This is the one category of the entire plan with zero flexibility. A store rejection or a legal gap doesn't slow the pilot down — it stops it outright, and several of these items are irreversible once skipped.

- Rename the app identifiers before any build reaches any store — `com.anonymous.customer-app` becomes `in.kyapehnu.app`, and the Expo slug moves off `customer-app`. Bundle identifiers are effectively permanent once published; changing one later means losing every existing install and starting the store listing over.
- EAS build profiles, an OTA update channel for JS-only fixes, and a signing-key backup — losing the Play upload key is unrecoverable without Play App Signing enrolment, so that enrolment happens up front.
- Legal pages live at public URLs and linked in-app: Terms of Service, Privacy Policy, Refund & Cancellation Policy, Shipping Policy, and a named grievance officer with contact details — required by Indian e-commerce rules, checked by Razorpay during onboarding, and checked again by both app stores.
- In-app account deletion plus the matching public web page — mandatory on both stores.
- Store assets: screenshots, a short preview video (the Thread and the Unfurl are the two moments worth showing), the Play data-safety form and Apple privacy labels filled out to match actual behaviour, and a seeded demo account so a reviewer never lands in an empty city.
- **Remove the Vendor Mode debug toggle** from the Profile screen, or gate it hard behind a real vendor account. A reviewer who flips into an empty order desk reads the whole app as unfinished.

9. Deferred to manual operations, with rationale

Nothing in this table is cancelled. Each row is a piece of automation replaced, for the pilot only, by a person doing the same job by hand — and each row states exactly why that trade is favourable at pilot volume.

Deferred	Manual replacement	Why this is the right trade now
WhatsApp listing bot	Admin creates the first ~20 listings per vendor by hand	The plan's own onboarding guidance already calls for this; the bot's first real user is listing #21.
Full escalation ladder (SMS, IVR, auto-reassign)	Push + WhatsApp alarm, then an ops person calling the vendor directly	At 20–40 orders a day one attentive person outperforms an automated ladder — and their phone calls reveal the real failure modes the automated version should handle.
Porter quote / track / webhooks / live map	Manual dispatch through Porter's consumer app; a tracking link pasted into the order board	Skips Porter's 2–4 week commercial onboarding entirely; the interface is still built so this is a same-shape adapter swap later, not a rewrite.
Vendor ledger, settlement runs, netting	A weekly spreadsheet	Fifty vendors, once a week, by hand is faster to build and easier to audit than a settlement engine with no real transaction history to validate it against.
Vendor KYC pipeline	Paper documents plus a couple of form fields	The verification depth of a real KYC flow is only worth building once vendor volume makes manual review the bottleneck.
Coupon engine, referral program, loyalty tiers	One hardcoded first-order offer	A configurable promotions engine has no promotions to configure yet.

Deferred	Manual replacement	Why this is the right trade now
Return / exchange flow	A phone call plus a manually arranged reverse delivery	Low volume makes a person a perfectly adequate router; the return *rate* is still tracked (Section 11) even though the flow isn't automated.
Pricing-rules UI, merchandising, feature flags, analytics dashboard	Config values in code, PostHog for analytics, a spreadsheet for merchandising calls	None of these need a UI until someone other than an engineer is operating them daily.
Support ticket queue	A phone number and a WhatsApp link; the grievance contact is the static legal page	A ticketing system solves a queueing problem the pilot doesn't have yet.

10. Deferred outright

These items have no manual substitute at launch — they simply wait, because either the customer base or the catalogue depth they depend on doesn't exist yet.

10.1 Customer-facing polish

- Search, filters, sort, and category browsing.
- Saved-address book UI (the backend already stores addresses; nothing reads them back yet).
- Reorder, and ratings/reviews for both products and shops.
- Dedicated shop profile pages, wishlist, and follow-a-shop with drop alerts.
- Customer-facing Hindi/Marathi localisation.
- A full dynamic-type and contrast accessibility audit beyond the basics already covered in Section 3.

10.2 Motion and identity

- The three remaining signature animations — the Rail, the Fold, the Window Light — and the full component gallery / Chapters tab.

10.3 Reliability polish

- Circuit breakers and formally defined degraded modes per external dependency — valuable at scale, premature when there are three external dependencies total and an ops person watching all of them directly.

10.4 All eight differentiator features

The “Kya Pehnu?” answer engine, Try Rack, fit profile, wardrobe, occasion / shared carts, Nagpur Now, and emergency mode all defer past the pilot, without exception. This is the least comfortable cut on the list and the most important one to hold: these are retention features, and a pilot with fifty shops and no purchase history has nobody to retain yet. Worse, several of them actively backfire at pilot scale — the answer engine specifically would return thin, repetitive results across fifty shops and read as broken rather than impressive. They are sequenced back in as soon as the pilot proves the core loop; see Section 17.

11. Capture the data now, even though the feature waits

This is the section that makes deferral safe rather than merely convenient. A handful of fields and events are nearly free to capture starting with order #1, and every one of them is the direct input to a feature or a decision that cannot be reconstructed retroactively once the pilot is underway.

Capture from order #1	Why it can't wait
Garment measurements (chest, length, shoulder) at listing time	Fit profile and Try Rack (Section 17) depend entirely on this. Impossible to backfill onto a garment nobody measured.
Occasion and formality tags on every product	The direct input to the “Kya Pehnu?” answer engine's lookup table. Cheap to add at listing time, expensive to re-tag an entire retroactive catalogue.

Capture from order #1	Why it can't wait
Actual delivery cost alongside the delivery fee charged, on every order — even under manual Porter dispatch	This single comparison is the number that decides whether the unit economics of the business work at all. It has to exist from the first order, not from the first order after the logistics integration ships.
Cancellation and complaint reasons as fixed enums, not free text	Free text written today cannot be aggregated later. An enum can be analysed the moment there are ten of them.
Product analytics events (PostHog) defined against the funnel described in the team's economics plan	A metric added after launch has no history, and a metric with no history carries no judgement about what changed.

12. The multi-vendor cart decision

The current cart store accepts items from any shop; the order schema stores a single vendor. This mismatch has to be resolved before Phase 1 begins, because it changes the shape of the order schema, and the schema is touched by payments, refunds, tracking, and payouts simultaneously.

Resolution: split now, gate narrow.

- Model a Checkout parent with N per-vendor Order children in the schema from the start.
- In the UI, gate it to effectively single-vendor behaviour at launch — either restrict a cart to one shop, or allow a second shop only when it's within a tight radius of the first, keeping a single delivery leg workable under manual dispatch.
- The additional engineering cost of the correct schema now is roughly a week. Retrofitting it after launch means touching live payment records, live refund logic, live tracking state, and live vendor payouts all at once — a materially riskier piece of work to do on a system that already has real money moving through it.

Model it right. Ship it narrow.

13. External dependencies — what to skip, what to start today

The original plan started seven external onboarding processes in week one. Most of that was unnecessary caution. Revised, only three genuinely block a hard date; everything else can start when the phase that needs it starts.

13.1 Start today — the only three with a hard deadline

Start now	Why it can't wait
Google Play — register as an organisation , not a personal account	A personal account requires a closed test with 12+ opted-in testers across 14 continuous days before production access is granted — an unavoidable two-week tail at the end of the project. An organisation account is exempt. This decision is effectively irreversible once made, so it is made in week one.
Apple Developer (organisation) — the D-U-N-S number specifically	A D-U-N-S lookup or registration can itself take days to weeks and gates the entire Apple Developer enrolment behind it.
Entity registration and GST, if not already in place	Gates Razorpay's production onboarding (roughly 3–7 days once GST exists) and is itself a multi-week process.

13.2 Explicitly skipped or deferred

Item	Disposition and reasoning
SMS / DLT registration	Cut from the plan entirely, not deferred. DLT is a 1–2 week regulatory registration (with your business as the principal entity, so no provider — including Twilio — can skip it on your behalf) for a channel that fails for the same reason push and WhatsApp already fail: the phone was in a pocket. The escalation ladder's working rung is the voice call, not SMS, so SMS is removed rather than worked around (Section 14).

Item	Disposition and reasoning
Porter production API access	Deferred to well past the pilot. Manual dispatch through Porter's own consumer app is the launch-day mechanism (Section 5.2), so the 2–4 week commercial onboarding no longer sits on the critical path at all.
Meta WhatsApp Business verification	Not skipped, but not urgent. All Phase 0–period development happens against Meta's own free test number, which requires no business verification and behaves identically to production for coding purposes. Start the real verification whenever convenient before Section 17's WhatsApp bot work begins — it no longer blocks anything before that.
Razorpay production keys	Test keys are sufficient for all of Phase 0–1. Production keys are only needed once GST and the legal pages (Section 8) exist — naturally around Section 15's later weeks.
Twilio, considered and rejected as a shortcut	Twilio is a reseller (“BSP”) of the same Meta Cloud API already integrated — it does not remove the underlying Meta business verification requirement, it adds a per-message markup on top of it, and adopting it would mean rewriting already-working code for no benefit. For voice specifically, an India-focused provider such as Exotel is the better call over Twilio: a foreign caller ID measurably lowers answer rates, and for a robocall whose entire job is to be answered, that matters more than convenience.

14. The escalation ladder, resolved

A 45-minute promise fails the moment a vendor doesn't notice a new order. The original design used five rungs (push, WhatsApp, SMS, automated voice, auto-reassign); with SMS cut (Section 13.2), the pilot ladder is four, and the last rung is a person rather than automation — which is both simpler to build and, at pilot volume, more reliable.

Timing	Channel	Detail
T+0s	Push notification + in-app alarm	High-importance channel, looping sound, full-screen swipe-to-accept card.
T+0s	WhatsApp utility message	Order id, item count, value, with an Accept quick-reply. Sent via Meta's own test number during development, production number once verification lands.
T+90s	Automated voice call (IVR)	"New order at your shop. Press 1 to accept." The rung that actually works — it interrupts a pocketed phone.
T+3min	Ops calls the vendor directly	Replaces the automated auto-reassign/cancel rung for the pilot; a human decides whether to wait, reassign to another shop holding the same stock, or cancel with a refund.

The same shortened ladder applies to "order marked ready but not yet picked up" and "rider waiting at the shop," both watched from the admin order board rather than automated at pilot volume.

15. Week-by-week phase plan

Roughly 10–12 weeks for one to two full-time engineers, to a live pilot in four Nagpur neighbourhoods. Phases are sequential where they must be — Phase 1 is the foundation everything downstream reads from — and explicitly parallel where they aren't.

Phase	Weeks	Scope
0	1	Fix the order-read authorisation hole and the unrestricted-field vulnerability first, in isolation. Then: validation, rate limits, CORS, env validation, CI, and the app-identifier rename. Start the three external registrations from Section 13.1 in parallel.
1	3	The foundation. Real auth (OTP + Google), the full customer API surface, deletion of the mock data layer, server-computed pricing, transactional stock, Razorpay (test keys) with COD, the address flow with serviceability checking, and the manual-adaptor logistics interface.
2	1.5	The shortened escalation ladder (push → WhatsApp → IVR → human), the vendor alarm channel, and the admin order board — the tool ops actually works from once orders are real.
3	1	Admin QC queue and the catalogue editor with image upload — the tool that creates a vendor's first twenty listings by hand.
4	1	The typography and motion token files, the Unfurl and the Thread animations, 3D scene device-tier gating, and the copy pass — run in parallel with Phase 3 by a second person if one is available.

Phase	Weeks	Scope
5	1.5	Hardening and compliance: the pricing/auth/transition test suite in CI, backups with a tested restore, legal pages, store assets, demo accounts, and removing the Vendor Mode debug toggle.
6	1–2, then ongoing	Pilot launch. Onboard 30–50 shops across Dharampeth, Sitabuldi, Ramdaspath, and Civil Lines — supply before any marketing spend. Ops runs the order board by hand for the first ~500 orders and keeps a written log of every failure; that log becomes Section 17's priority order, replacing guesswork with evidence.

Design (Phase 4) and WhatsApp/admin work (Phases 3) are deliberately timed to start once the data shapes from Phase 1 are real — building either against mock data is expensive rework waiting to happen.

16. Risk register

Risk	Likelihood	Impact	Mitigation
Manual Porter dispatch doesn't scale attention past a handful of concurrent orders	Medium	High	Deliberately the trigger for building the Porter API adapter (Section 17) — the ops log from Phase 6 defines exactly which manual steps to automate first.
Vendors don't adopt the platform without the WhatsApp bot's convenience	Low at launch	Medium	Irrelevant at pilot scale — admin does the listings directly; adoption of self-service listing is measured post-pilot, once the bot exists.
45-minute promise missed regularly	Medium	High — this is the brand	Four neighbourhoods, not a city; an honest live-order status rather than a simulated map; automatic compensation on a miss.
Return rate erodes contribution margin	Medium	High, but slow-moving	Fit-relevant data captured from order #1 (Section 11) even though fit profile itself waits, so the eventual feature launches with real measurements behind it rather than a cold start.
Single engineer, ~11-week schedule	Medium	Schedule slip	Phases 3 and 4 are explicitly parallelisable — that is where a second person adds the most value if one becomes available.
Google Play account type chosen wrong	Low if decided now	Two-week delay at the worst possible time	Resolved in Section 13.1 — organisation account, decided in week one, before it can become a problem.
The order-read authorisation hole is exploited before it's patched	Low, but the fix is trivial	Customer data exposure	First commit of Phase 0, before any other engineering work begins.

17. Post-pilot: what gets built next, and in what order

Everything deferred in Sections 9 and 10 gets built — the pilot's job is to decide the order, using evidence instead of the guesswork any pre-launch sequencing necessarily relies on. The written failure log from Phase 6 is the actual input to this list; the ordering below is a reasonable starting hypothesis, not a fixed schedule.

- **Automate whichever manual-ops item is consuming the most attention.** If ops is spending most of its time on the phone chasing Porter dispatch, the Porter API adapter goes first. If it's chasing vendors, the escalation ladder's SMS/IVR rungs and auto-reassign logic go first. The pilot log answers this question; no amount of upfront analysis can.
- **The WhatsApp listing bot**, once enough vendors have outgrown admin-assisted onboarding that it's become the bottleneck rather than a convenience.
- **Fit profile and Try Rack**, once there is enough order history and enough garment-measurement data (captured from day one per Section 11) for a fit system to be accurate rather than a guess dressed up as a feature.
- **The “Kya Pehnu?” answer engine**, once the catalogue is thick enough across enough shops that a three-tap query returns genuinely good options rather than the same five items repeated.

- **The remaining admin modules** — vendor KYC pipeline, pricing-rules UI, coupon builder, merchandising, analytics dashboard — as the spreadsheet each currently replaces starts to visibly strain under real volume.
- **Wardrobe, occasion carts, Nagpur Now, emergency mode, referral and loyalty mechanics** — the retention layer, once there is a base of repeat customers worth retaining.
- **The remaining three signature animations** (Rail, Fold, Window Light), the Chapters tab, and the full accessibility and localisation passes, as the app matures from proving the core loop works to competing on how it feels to use every day.

The moat this plan is built to prove is not the software. It is fifty shopkeepers in Nagpur who will keep their stock accurate, and a rider willing to wait ten minutes while a customer tries something on. Everything in this document exists to get to that proof as fast as honestly possible, and to make sure nothing built along the way has to be thrown away once it arrives.